

Cyber security and beyond: Detecting malware and concept drift in AI-based sensor data streams using statistical techniques[☆]

Muhammad Amin^{a,*}, Feras Al-Obeidat^b, Abdallah Tubaishat^b, Babar Shah^b, Sajid Anwar^c, Tamleek Ali Tanveer^c

^a National University of Computer and Emerging Sciences (NUCES-FAST), 25000, Pakistan

^b Zayed University, United Arab Emirates

^c Institute of Management Sciences, 25000, Pakistan

ARTICLE INFO

Keywords:

IoT
Concept drift
Statistical methods
SVM
Malware detection

ABSTRACT

In the Industrial Internet of Things (IIoT), mobile devices can be used to remotely monitor and control industrial processes, equipment, and machinery. They can also be used to optimize production and maintenance processes, improve safety, and increase efficiency in industries such as manufacturing, energy, and transportation. The adoption of IIoT has the potential to increase production and efficiency, but it also raises new cybersecurity concerns since interconnected industrial systems are more susceptible to malware intrusions. Malware attacks on IIoT systems can have grave consequences, including production delays, data loss, and physical asset damage. To aid this we propose to use statistical drift detection methods to perceive any change in data patterns and train the machine learning classifiers to counter newly developed malware samples then and there. Our results with an accuracy of 95.2% and F1-score of 94% indicate that our approach is highly successful and easy to adopt.

1. Introduction

Artificial intelligence (AI)-based sensors evaluate and interpret data gathered from the environment using AI algorithms. The Industrial Internet of Things (IIoT) and mobile devices are only two examples of the sectors that employ these sensors. The “Internet of Things” (IoT) is a phrase used to describe how sensor nodes, such as home and industrial equipment, are linked to one another through the internet to allow for remote data gathering, data sharing, and control. IoT has been used more and more in recent years, and it has the potential to change many parts of our everyday life. The industrial Internet of Things (IIoT) refers to the integration of IoT technology in industrial processes and systems, such as manufacturing, energy, and transportation. Since 2018, there has been an annual increase in global IoT investment of at least \$40 billion. Moreover, spending is anticipated to expand to \$1.1 trillion in 2023, maintaining the accelerated rate of growth each year [1]. The global spending details are depicted in Fig. 1.

However, in 2021–22, the number of malware attacks on IoT devices increased by nearly 60% compared to the previous year, according to a report by Kaspersky [2]. In addition, a study by the cybersecurity firm McAfee [3] found that the number of unique malware families targeting IoT devices increased by almost 50% in 2020, with a total of 1,893 families of malware detected.

Nothing is static in the changing world. When it comes to the IoT data, this is especially true. The underlying distribution of the data derived from a verified source evolves with time.

[☆] This paper is for special section VSI-siot. Reviews were processed by Guest Editor Dr. Deepak Gupta and recommended for publication.

* Corresponding author.

E-mail addresses: muhammad.amin@nu.edu.pk (M. Amin), Feras.Al-Obeidat@zu.ac.ae (F. Al-Obeidat), Abdallah.Tubaishat@zu.ac.ae (A. Tubaishat), babar.shah@zu.ac.ae (B. Shah), sajid.anwar@imsciences.edu.pk (S. Anwar), tamleek@imsciences.edu.pk (T.A. Tanveer).

<https://doi.org/10.1016/j.compeleceng.2023.108702>

Received 7 January 2023; Received in revised form 21 March 2023; Accepted 23 March 2023

Available online 30 March 2023

0045-7906/© 2023 Elsevier Ltd. All rights reserved.

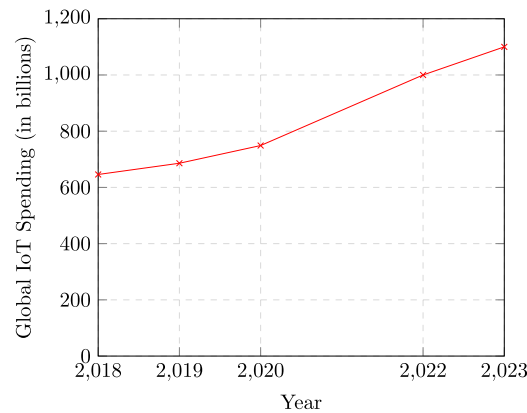


Fig. 1. Global IoT spending [1].

Consider a business system for product recommendations as an illustration. Do you believe a model developed before COVID-19 would perform just as well during the COVID-19 pandemic? These types of unplanned events resulted in significant changes in consumer behaviors. The majority of customers prefer buying inexpensive necessities over pricy luxuries. Thus, the data on user behavior has altered. Along with that, one can observe a completely new type of shopping behavior for customers because many things are out of stock in the market under this kind of circumstance [4].

When these data changes result in the learning model built using the old data being conflicting with the new data, this is an issue called “Concept Drift”.

There are several different types of statistical models that can be used to detect and adapt to concept drift [5]. A change point detection algorithm is a popular method for determining when the statistical characteristics of the data have considerably changed. This algorithm employs statistical tests to make this determination. Another approach is to use a sliding window method, where the algorithm with a fixed-size window is trained on the most recent data, and the window is slid forward as new data arrives. This enables the model to adjust as the distribution of the data does. Utilizing an adaptive model, such as an online learning algorithm, is an additional strategy.

Retraining is the process of upgrading a machine learning model by training it on new data, according to [6]. The functionality of the model may be improved, the data distribution may be adjusted, or new information or experience may be added to the model. The method and hyperparameters for the original model can be used for retraining, or the algorithm or hyperparameters can be modified to better fit the new data. Retraining is a standard procedure in machine learning and may be carried out on a regular basis, whenever the model’s performance starts to deteriorate or whenever new information or data becomes available.

Transfer learning [7] is the process of using techniques or information gained from one domain or job to another. Because the pre-trained model can serve as a suitable starting point for the new model, this can be helpful when it is impracticable or would take a huge amount of data to train a new model from begin. Because the pre-trained model’s expertise may be applied to the new task and help the new model learn more quickly and precisely, transfer learning can improve the new model.

Transfer learning and retraining, which both entail updating a machine learning model by training it on new data, have similarities with each other [6,7]. The two strategies do differ in several significant ways, though. Retraining includes updating a model by training it on the same task using new data, whereas transfer learning involves training a new model for a different problem using a previously trained model for a similar task. While retraining is usually done to improve a model’s performance on the same task or to adapt to changes in the data distribution, transfer learning can be helpful when training a new model from scratch is impracticable or would need a huge quantity of data. One particular type of retraining is transfer learning, where the newly trained model is given a head start by the previously trained model.

An online classifier for IoT (Internet of Things) is a machine learning model that can classify data streams in real-time as they are generated by IoT devices. Online classifiers are useful for IoT applications because they can process data quickly and make decisions based on the latest information available, which is important in dynamic environments such as the IoT.

Problem: Fig. 2 is showing the results of a process called “drift adaptation”, where a machine learning model is continuously updated with new data in order to maintain its performance. Drift adaptation is a way to deal with the fact that the distribution of data can change over time, which can cause a model’s performance to degrade [8]. In this case, it shows that the model was able to recover its perfect performance by retraining the new data based on the final data decision boundary and rebuilding the model using this updated labeling. This suggests that the model was able to effectively adapt to the changing data distribution and maintain its performance. We see that a machine learning model that was trained on a dataset with a clear decision boundary at time t_1 , and that the model was able to accurately classify new data at this time. However, as time goes on and understanding of the data evolves, the decision boundary we use to label new data may change, leading to a discrepancy between the model’s decision boundary and the true decision boundary of the data. In this case, it is likely that the model’s predictions for the new data will be inaccurate in the areas where the model’s decision boundary does not agree with the true decision boundary. This is because the

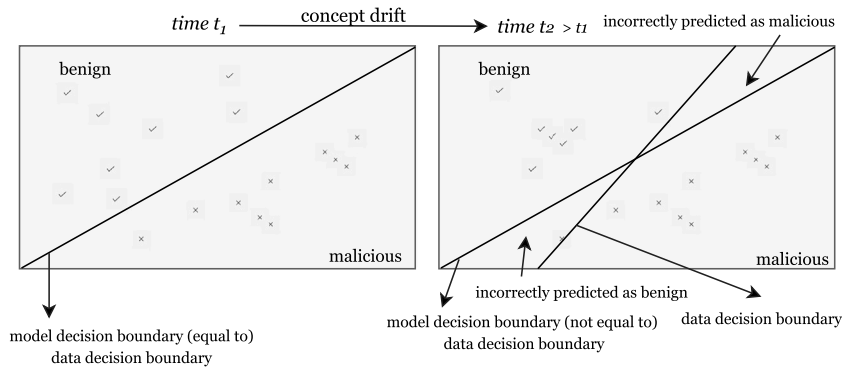


Fig. 2. An example of concept drift with retraining.

model was not trained on this data and does not have the necessary information to accurately classify it. One might need to retrain the model using the updated labeling to match the decision boundary of the model with the actual decision boundary of the new data in order to enhance the model's performance on the new data.

Although there has been a lot of interest in the detection of changes in online data streams, most of it depends on modeling the statistical properties of examples, which calls for a sizable sample size.

Objectives:

1. Reduce the human effort required for concept drift detection by applying statistical approaches,
2. And grant computers the task of training to enhance the discriminative features in data quickly and consistently to make predictions about malware in IoT that are drift aware.

Our main goal is to identify changes rapidly and precisely so that the classifier can reduce cumulative prediction error. Detecting underlying changes is one approach of preventing concept drift. Real-world domains where detecting changes in early time is important include data modeling, tracking of AI-Based industrial sensors and biological processes, defect identification, and diagnostics [5].

Contribution:

The fact that the target concept's underlying distribution could vary over time presents a challenging issue in learning contexts. Typically, this is referred to as Concept Drift. When a Malware classifier successively receives training examples in an AI-Based online learning environment, we propose to use statistical methods [5,8] to detect concept drift in the employed IoT datasets. And then classify them with machine learning algorithm. We are sampling a portion of the new data in each batch and evaluating the model's performance using the f1-score. If the f1-score falls below an acceptable level or statistical model suggests retraining, classifier automatically trigger a process to re-label the data and retrain the model. As a result, the model can sustain performance while adapting to modifications in the distribution of data.

These different sorts of concept drift methods are computationally feasible and precise. we empirically demonstrated the each method's accuracy and outcome.

Paper Organization: Section 2 summarizes and synthesizes the existing research and include a critical analysis of the strengths and limitations of the previous research. Section 3 of research paper describes the research method, data collection and analysis methods used in the study. It gives enough detail for other researchers to replicate the study. The results, Section 4, of the paper presents the findings of this study. In conclusion Section 5, the summary of the main findings of the study and their implications are given. It also includes recommendations for future research.

2. Literature review

In an online classifier as new data becomes available, a form of machine learning system, is trained and incrementally updated. In contrast, batch learning trains the model on the full dataset and does not allow for model updates until the model is retrained on the complete dataset.

Online classifiers are well-suited to applications where the data is constantly changing, such as in real-time streaming data (IIoT), because they can be updated as new data arrives and adapt to changes in the data distribution. Online classifiers are typically more efficient and can handle larger datasets than batch learning algorithms, but if the data distribution dramatically varies over time, they can be more prone to concept drift.

In [9] authors gives an summary of the various approaches. Concept drift, which is the change in the statistical characteristics of a data stream with time, is detected and accommodated using these methods. The paper covers both online and offline approaches to concept drift adaptation, and it discusses the strengths and limitations of each approach.

Table 1
Summary of literature review.

Paper	Approach	Pros	Cons
[9]	Online and offline approaches to concept drift adaptation	Covers a wide range of approaches	Limitations of each approach not fully discussed
[10]	Statistical, probabilistic, and machine learning-based methods for concept drift detection	Wide range of approaches covered	Limitations of each approach not fully discussed
[11]	Statistical, probabilistic, and machine learning-based methods for online concept drift detection	Wide range of approaches covered	Limitations of each approach not fully discussed
[12]	Methods for detecting concept drift in imbalanced data streams	Covers a wide range of approaches	Limitations of each approach not fully discussed
[13]	comprehensible macro- and micro-visualization for early concept drift detection	Accurate and efficient detection of concept drift	Limited to certain types of data streams
[14]	Kernel density estimation for concept drift detection	Accurate detection of concept drift	Limited to certain types of data streams
[15]	Comparison of various concept drift detection methods for data stream classification	Wide range of benchmarks and datasets used for evaluation	Limitations of each approach not fully discussed
[16]	Classification of concept drift detection techniques into three categories	Recommendations for future research provided	Limitations of each approach not fully discussed
[17]	Classification of concept drift detection techniques into four categories	Recommendations for future research provided	Limitations of each approach not fully discussed
[18]	Overview of methods for adapting to concept drift in data streams	Covers a wide range of approaches	Limitations of each approach not fully discussed

In [10] authors cover a wide range of approaches, including statistical, probabilistic, and machine learning-based methods, and it discusses the advantages and disadvantages of each approach.

In [11] authors review the various methods that have been designed for identifying concept drift in online data streams. The paper covers a wide range of approaches, including statistical, probabilistic, and machine learning-based methods, and it discusses the strengths and limitations of each approach.

In [12] authors review the various methods that have been designed for evaluating in imbalanced datasets. The paper covers a wide range of approaches, including statistical, probabilistic, and machine learning-based methods, and it discusses the challenges and limitations of each approach.

In [13] authors suggest a novel method based on a comprehensible macro- and micro-visualization for identifying concept drift in data streams. The authors show that their methodology can identify idea drift more precisely and effectively than existing approaches, and they go over the possible uses of their method in several fields.

In [14] authors provide a novel kernel density estimate method for identifying concept drift. The authors show that their methodology can successfully identify idea drift in a variety of data streams, and they go on to explore how their method may be used in fields like machine learning and data stream processing.

In [15] authors evaluate the methods using a variety of benchmarks and datasets, and they discuss the strengths and limitations of some approaches. The use of open source Big Data technology for data stream mining is also discussed.

In [16] authors discuss the challenges of drift in datasets and classify the techniques into three categories: supervised learning, unsupervised learning, and hybrid approaches.

In [17] authors give a survey of the various techniques classify them into four categories: supervised learning, unsupervised learning, hybrid approaches, and meta-learning approaches.

In [18] authors give a summary of the many strategies that have been created to deal with idea drift in data streams, including methods for identifying concept drift, adapting for concept drift, and fusing both strategies.

More recent approaches like [19–21] can provide a better understanding of concept drift an malware detection in bytecode obtained from datasets. There are several ways to minimize cumulative prediction errors during online learning. One approach is to use a model with a high capacity, such as a deep neural network, which can learn complex patterns in the data and make more accurate predictions. Using regularization techniques, such as weight decay or dropout, is another strategy that can assist prevent overfitting and enhance the model's generalization ability. Another approach is to use an online learning algorithm that can adapt to changes in the data distribution, such as an adaptive boosting or online gradient descent algorithm. These algorithms can automatically adjust the model parameters as new data arrives and reduce cumulative prediction errors over time. Table 1 provides a brief summary of all the discussed literature.

3. Proposed statistical drift detection for IIoT malware detection

The Proposed methodology [5] used in this work is depicted in Fig. 3 and discussed in the same order down below:

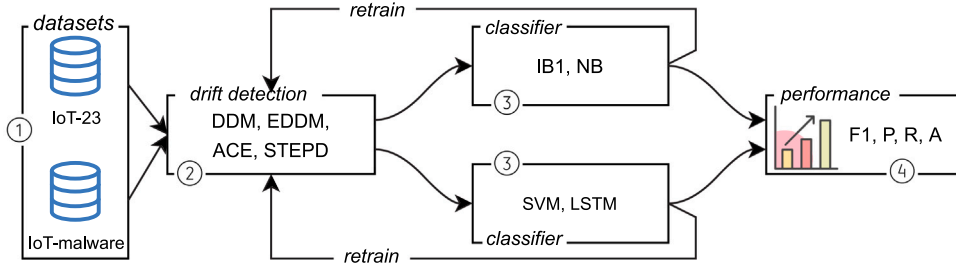


Fig. 3. Flow diagram for proposed methodology.

3.1. Datasets

These datasets are carefully chosen, since these are vastly used and cited in reputable avenues. These are freely available and currently active datasets. Where ever necessary we used input compression and pre processing techniques. Also, we used standard 10-Fold Cross Validation.

IoT-23: The IoT-23 [22] dataset was created by researchers and contains 23 different types of Internet of Things (IoT) malware. The dataset includes a variety of different malware types, including botnets, worms, and trojans, and is designed to be representative of the types of malware that are commonly found in the wild.

IoT Malware: IoT Mlaware [23,24] have used an extensive collection of Iot Malware. The authors of this work used the Debian installer bundle and the Object-Dump tool to extract application packages and decompile all samples. They then extracted the opcode sequences from the decompiled files [20]. Additional information about the IoT samples can be found in the CyberIOCs.

3.2. Drift detection method

The drift detection method (DDM) [25] with an online classifier is a technique for detecting changes in the target concept in an online learning scenario. It discusses the concept of error rate and its standard deviation in the context of a decision model predicting binary outcomes (True or False). The formula presented is used to compute the standard deviation of the error rate, which is defined as the likelihood of seeing a false prediction. This information can be used to evaluate the accuracy and stability of the decision model over time. The probability of seeing False at each point in the sequence, p_i , is used to calculate the error rate, with the standard deviation being determined by

$$s_i = \sqrt{\frac{p_i \{1 - p_i\}}{k}} \quad (1)$$

The error rate, which is the likelihood of seeing a false prediction, is described by this equation at each point in a sequence of samples. The number of errors in a sample of n instances, which follows a binomial distribution, is used to compute the chance of detecting a false prediction, represented as p_i . The standard deviation of the error rate, denoted as s_i , is then calculated as the square root of the probability of observing a False prediction multiplied by the probability of not observing a False prediction, all divided by the number of examples in the sample (i). This standard deviation can be used to measure the uncertainty or variability of the error rate at each point in the sequence.

When DDM detects a potential change in the concept, it stores examples in short-term memory and rebuilds the classifier using these examples. This allows the model to adapt to the new concept and improve its predictions.

3.3. Early drift detection method

The early drift detection method (EDDM) [26] is used for detecting gradual changes in the target concept in an online learning scenario. It operates by continually tracking the amount of time that passes between two categorization errors and utilizing this data to assess if the concept has shifted.

The uncertainty in the time interval is taken into consideration by EDDM by calculating the average distance between two errors, p'_i , and its standard deviation, s'_i . The classifier is rebuilt using the examples that were stored in short-term memory when EDDM noticed a probable change in the notion. As a result, the model may adjust to the novel concept and produce better forecasts.

EDDM is effective at detecting gradual changes in the concept, but may not perform well in noisy examples.

3.4. Adaptive classifiers-ensemble

A method for spotting shifts in the target concept in an online learning situation is the adaptive classifiers-ensemble system for concept-drifting environments (ACE) [27]. It utilizes an online classifier and evaluates the classifier's predicting performance using the most recent W samples. The classifier is initialized if $qt < qt - W^l$, where $qt - W^l$ is the smaller outcome of the confidence interval at time $t - W$.

3.5. Detection method using statistical testing

Detection Method Using Statistical Testing (STEPD) [5] is a technique for quickly and accurately detecting changes in the target concept in an online learning scenario. It uses two accuracy values: the recent accuracy and the total one. According to STEPD, a classifier's accuracy for recent W samples will be equal to the overall accuracy from the beginning of learning, and a significant decline in the recent accuracy indicates that the target concept is changing.

STEPD compares the recent and overall accuracy scores using a statistical test, then computes the P -value to display the observed significance level. If the P -value falls below a specified significance level, the null hypothesis is rejected and the alternative hypothesis (that the concept is changing) is accepted (that the recent and overall accuracy are equal). The detection is carried out by computing the statistic shown below:

$$T(r_o, r_r, n_o, n_r) = \frac{|r_o/n_o - r_r/n_r| - 0.5(1/n_o + 1/n_r)}{\sqrt{\hat{p}(1-\hat{p})(1/n_o + 1/n_r)}} \quad (2)$$

This equation is an expression for, and similar to the McNemar's test statistic, which is a statistical test used to compare the performance of two binary classifiers. It is often used to determine whether the performance of one classifier is significantly different from another classifier.

In this equation, $T(r_o, r_r, n_o, n_r)$ represents the McNemar's test statistic, and r_o and r_r represent the number of correct and incorrect classifications, respectively, made by each classifier. n_o and n_r represent the total number of samples classified by each classifier. The term $|r_o/n_o - r_r/n_r|$ represents the difference between the proportions of correct classifications made by each classifier.

3.6. Classification algorithms

IB1: IB1 (Instance-Based 1-Nearest Neighbor) is a type of lazy learning algorithm, that do not build a model upfront, but instead learn from the data on-the-fly. The IB1 algorithm works by storing all of the training data in memory, and then for each new instance, it finds the training instance that is most similar to the new instance (i.e. the nearest neighbor) and uses that instance to make a prediction. It is easy to use and may be applied to a variety of categorization applications. It could not function effectively with datasets that have a significant percentage of duplicated or unneeded features since it is sensitive to data noise.

Naive Bayes: Based on the Bayes theorem of probability, Naive Bayes is a probabilistic machine learning method. Since it presumes that every characteristic in a dataset is independent of every other feature, which is frequently untrue in real-world data, it is known as a "naive" approach. Given this oversimplification, Naive Bayes might well perform well in many classification problems and is often employed in tasks like text categorization and spam filtering. The algorithm sorts the input into the class with the highest probability after computing the probability of each class given the properties of the input. Naive Bayes is a common choice for many machine learning problems since it is simple to use and rapid to train.

Support vector machines: An example of a supervised machine learning technique that may be applied for classification problems is support vector machines (SVMs). They function by identifying the hyperplane in a high-dimensional space that most effectively divides the various classes.

To train an SVM for classification, the algorithm is given a set of labeled training data, where each data point belongs to one of several classes. This training data is then used by the SVM to identify the hyperplane that best divides the various classes. The SVM may categorize new data points by identifying which side of the hyperplane they fall on after the hyperplane has been located. One advantage of SVMs is that they are able to handle high-dimensional data and non-linearly separable data. They are also relatively resistant to overfitting, meaning that they tend to perform well on unseen data. SVMs may need some fine-tuning to obtain acceptable performance, though, since they can be sensitive to the selection of hyperparameters, such as the kernel function and the regularization parameter.

Tuning these hyperparameters can be a challenging task, and it is often done using techniques such as cross-validation or grid search. Properly setting the hyperparameters can significantly improve the performance of an SVM model, but it can also be time-consuming and require a good understanding of the data and the problem at hand.

Long Short-Term Memory: Recurrent neural networks of the Long Short-Term Memory (LSTM) kind are frequently employed in time series forecasting and natural language processing. They are designed to remember long-term dependencies in the data and can be trained to recognize and predict patterns in sequential data, such as text, speech, or time series data.

In an LSTM (Long Short-Term Memory) network, gates are used to control the flow of information through the network. The gates are implemented using sigmoid activation functions, which produce a value between 0 and 1, and are used to determine which information to remember or forget at each time step. Input gates, forget gates, and output gates are the three types of gates that make up an LSTM network. The input gate determines which data from the input is sent on to the cell state. The forget gate determines which data from the previous cell state is retained in the current cell state. The output gate also controls the information from the cell state that is used to produce predictions. By controlling the information flow through the network with the help of these gates, LSTM networks are capable of learning long-term relationships in the data and learning to provide precise predictions based on these dependencies.

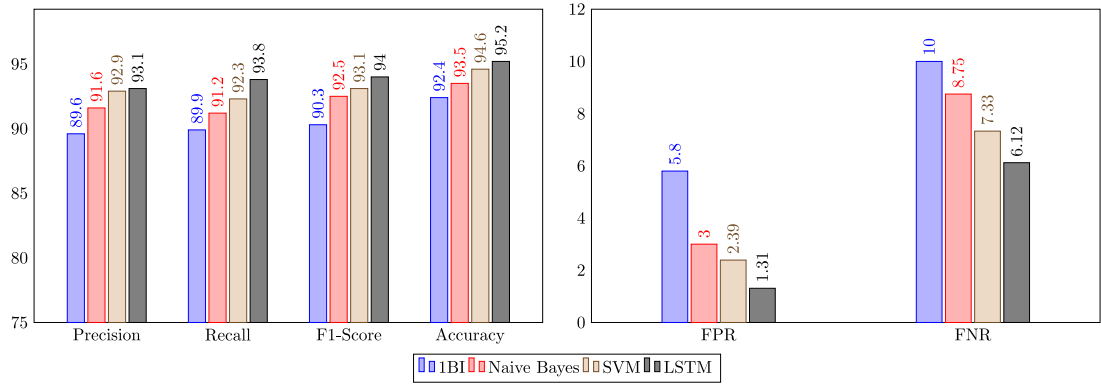
Let x_t be the input feature vector at time step t , and let h_t be the hidden state vector of the LSTM network at time step t . Then, the LSTM network updates its hidden state and computes the output as follows:

$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \quad (3)$$

Table 2

Results for drift detection in datasets used via employed statistical methods (IB1 and NB). Number of drift detection (N_d), number of instances needed to accurately identify drift (N_e), and cumulative prediction error rate with confidence interval.

Data	Method set	IB1			Naive Bayes (NB)		
		Error rate	N_d	N_e	Error rate	N_d	N_e
IoT – 23	STEPD	.0334 ± .0221	3.000 – .200	7.68	.0784 ± .0071	3.554 – .031	6.45
	DDM	.0653 ± .0084	3.670 – .113	10.42	.0287 ± .0012	3.500 – .370	8.94
	EDDM	.0714 ± .0110	3.400 – .002	25.6	.0208 ± .0000	3.220 – .021	22.1
	ACE	.0885 ± .0001	3.312 – .010	15.6	.0321 ± .0021	3.030 – .038	13.6
	Not Use	.9134 ± .0211			.3351 ± .0006		
IoT – Malware	STEPD	.2353 ± .0012	3.800 – .212	15.7	.5218 ± .0197	3.713 – 2.122	11.76
	DDM	.3124 ± .0215	4.124 – .1212	19.3	.7241 ± .0137	3.817 – 2.005	22.1
	EDDM	.4121 ± .0124	4.918 – .1356	28.2	.8550 ± .0117	3.799 – 2.998	22.0
	ACE	.3110 ± .0206	4.991 – .6124	17.1	.7399 ± .0213	3.910 – 2.334	12.99
	Not Use	.5127 ± .0345			.7837 ± .0220		

**Fig. 4.** Results when drift is detected and retraining is performed on IoT-23.

$$f_t = \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \quad (4)$$

$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \quad (5)$$

$$g_t = \tanh(W_{gx}x_t + W_{gh}h_{t-1} + b_g) \quad (6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (7)$$

$$h_t = o_t \odot \tanh(c_t) \quad (8)$$

Where,

$\sigma(z)$ is the sigmoid activation function, $\tanh(z)$ is the hyperbolic tangent activation function, W are the weight matrices, b are the bias terms, $\odot$ represents element-wise multiplication. The output of the LSTM network is usually computed by passing the final hidden state h_T through a fully connected layer with a sigmoid activation function, followed by a binary cross-entropy loss:

$$y = \sigma(W_{hq}h_T + b_q) \quad (9)$$

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \quad (10)$$

Where,

y is the predicted class probability, $\hat{y}$ is the ground truth class label, N is the number of samples in the batch, $\mathcal{L}$ is the binary cross-entropy loss.

4. Results

Evaluation Criteria: The Table 4 statistics are used for evaluation. In the Table 4, the letters TP, FP, TN, and FN stand for the number of true positive forecasts, true negative predictions, false negative predictions, and false positive predictions, respectively.

Table 3

Results for drift detection in datasets used via employed statistical methods (*SVM and LSTM*). Number of drift detection (N_d), number of instances needed to accurately identify drift (N_e), and cumulative prediction error rate with confidence interval.

Data	Method set	SVM			LSTM		
		Error rate	N_d	N_e	Error rate	N_d	N_e
IoT – 23	STEPD	.0299 ± .0111	3.110 – .011	6.389	.0076 ± .0002	1.998 – .010	4.89
	DDM	.0745 ± .0221	3.121 – .211	7.35	.0087 ± .0002	2.000 – .370	8.94
	EDDM	.0829 ± .0289	3.116 – .0216	27.3	.0208 ± .0000	2.000 – .000	42.0
	ACE	.0601 ± .0451	3.232 – .0314	11.4	.0100 ± .0002	2.000 – .008	11.4
	Not Use	.9134 ± .0211			.3351 ± .0006		
IoT – Malware	STEPD	.3568 ± .0707	4.124 – 1.015	9.8	.3114 ± .0435	4.019 – 1.002	8.91
	DDM	.4426 ± .0125	4.829 – .892	13.5	.4332 ± .0328	3.8990 – 1.023	13.0
	EDDM	.5959 ± .0227	4.972 – 1.123	24.2	.5798 ± .00824	4.916 – 3.129	18.0
	ACE	.4456 ± .0917	4.191 – 1.003	15.2	.3829 ± .0923	3.981 – 2.221	14.96
	Not Use	.5127 ± .0345			.7837 ± .0220		

Table 4

Evaluation metrics.

Metric	Formula
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
F1 Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
FPR	$\frac{FP}{FP+TN}$
FNR	$\frac{FN}{FN+TP}$

Table 5

Performance of different classifiers on the data.

Method	Precision	Recall	F1-Score	Accuracy
IB1	89.6	89.9	90.3	92.4
Naive Bayes	91.6	91.2	92.5	93.5
SVM	92.9	92.3	93.1	94.6
LSTM	93.1	93.8	94.0	95.2

The left plot in Fig. 4 compares the performance of four different models using the following evaluation metrics: precision, recall, F1-score, and accuracy. The right plot in Fig. 4 compares the false positive rate (FPR) and false negative rate (FNR) for the same four models.

Based on the results shown in Fig. 4 and Table 5, the LSTM model appears to best results for the given evaluation criteria. In comparison to the other three models, it has the greatest precision, recall, F1-score, and accuracy. Among the four models, the IB1 model has the lowest F1-score, accuracy, precision, and recall. The Naive Bayes and SVM models have intermediate performance, with the Naive Bayes model having slightly better precision and F1-score, and the SVM model having slightly better recall and accuracy.

In terms of the false positive and false negative rates, Among the four models, the LSTM model likewise has the lowest rates, demonstrating that it makes fewer inaccurate predictions than the actual number of positive and negative cases in the dataset. The IB1 model has the highest false positive and false negative rates, indicating that it has a higher number of incorrect predictions relative to the other models. The Naive Bayes and SVM models have intermediate false positive and false negative rates, with the Naive Bayes model having slightly lower rates and the SVM model having slightly higher rates.

In general, a model with a low FPR and a low FNR is considered to be performing well, as it is making a low number of incorrect predictions relative to the actual number of positive and negative cases in the dataset. However, the trade-off between FPR and FNR may vary depending on the specific application and the desired performance of the model.

The inefficiency brought on by irrelevant features is one of the main issues with nearest neighbor (NN) algorithms. Giving features weights based on how important they are for categorization is one way to address this issue. The three types of weighting techniques now used – global, partial, and local – along with approximate nearest neighbor search can be used to accurately reflect the relative importance of features for classification [28,29].

We see that online classifiers are well-suited to applications where the data is constantly changing, such as in real-time streaming data (Industrial IoT), because they can be updated as new data arrives and adapt to changes in the data distribution (see Table 5).

4.1. Drift attribution

Concept drift types are shown in Fig. 6 and includes:

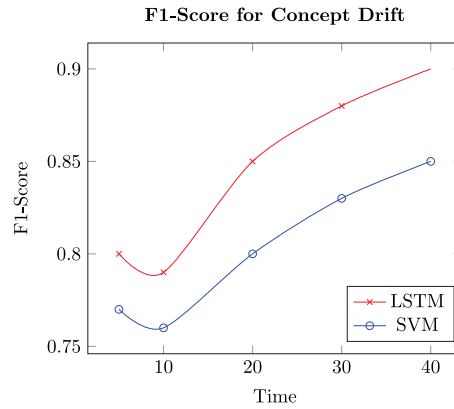


Fig. 5. Drift dip is detected and retraining is performed. The resultant rise in F1-score is plotted over time.

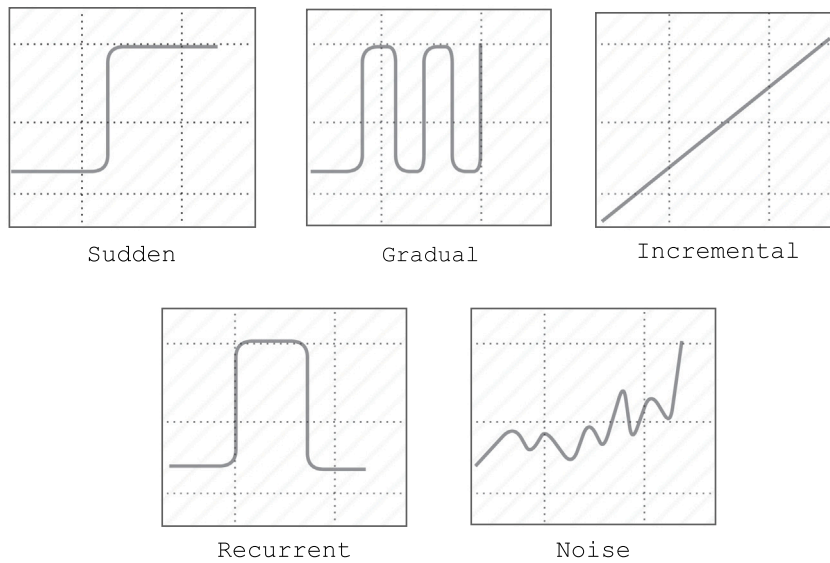


Fig. 6. Concept drift types.

Sudden: The transition from one concept to another occurs rapidly. The beginning of the COVID-19 public lockdowns in March 2020, which abruptly altered population habits all around the world, is a clear illustration of this. **Incremental/Gradual:** As the new concept arises and matures, there is a gradual shift between conceptions. An illustration of steady drift is the annual transition from summer to winter. **Recurring/Seasonal:** The changes continue to occur after the initial observation. The yearly seasonal weather change, which encourages customers to purchase thick jackets in colder months, then stops demand as temperatures rise in spring, and then starts again in the fall, is an illustration of this sort of drift. How drift is managed might vary greatly depending on how it manifests itself in a given use case. We note that IoT-23 dataset is characterized as sudden type. Where as IoT Malware dataset is characterized as gradual type with respect to concept drift.

4.2. Discussion

Fig. 7 shows retraining process is in place for continuously updating a machine learning model using new data. The model we have shown here is SVM classifier for a sample example, which is a type of supervised learning algorithm that can be used to classify data into different categories [8].

In this case, the model is being trained on data that follows a linear decision boundary, which is a line that can be used to separate different classes of data. The decision boundary is initially oriented at a 10 degree angle with the horizontal, and with each new batch of data, the decision boundary is rotated counterclockwise by 0.2 degrees.

We are sampling a portion of the new data in each batch and evaluating the model's performance using the f1-score, which is a measure of the model's accuracy. If the f1-score falls below an acceptable level or statistical model suggests retraining, classifier

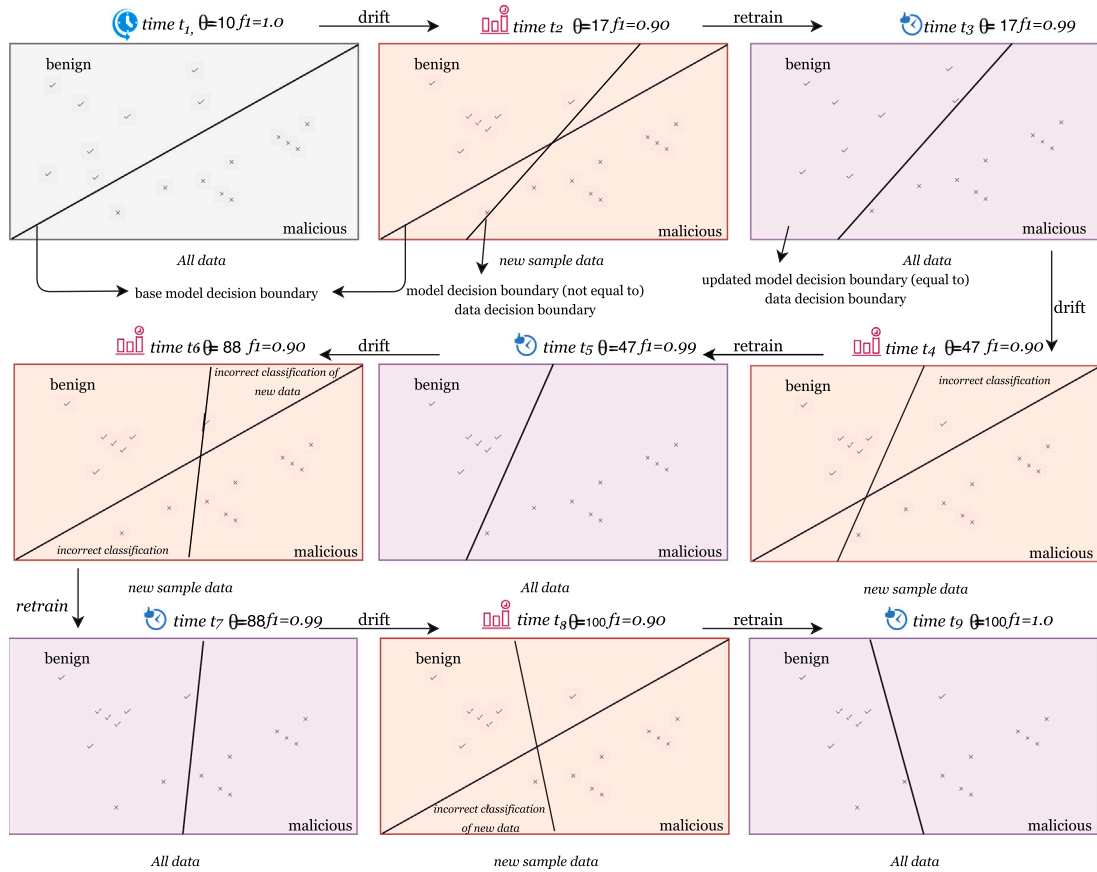


Fig. 7. This illustration shows the process of concept drift and then retraining the SVM with new samples which adjusts the SVM hyperplane and F1-Score is updated accordingly [8].

automatically triggers a process to re-label the data and retrain the model. This allows the model to adapt to changes in the data distribution and maintain its performance.

If sudden drifts in a particular situation can be reasonably anticipated, a concept drift detection system is employed as a critical part of the complete pipeline for a rapid adjustment in our features or projections. Yet if the same detectors were employed in the case of gradual drift, it is quite possible that they would not be able to detect the drift at all, and system performance would start to degrade. Based on the results shown in Fig. 5, both the LSTM and SVM models seem to be functioning well in terms of F1-score, as both models have obtained F1-scores of about 0.8–0.9 at various times. Particularly at later times, the LSTM model seems to have marginally higher F1-scores than the SVM model (e.g. 40 time units).

Results for drift detection from statistical methods are populated in Tables 2 and 3. We can see that DDM does a decent job of detecting sudden changes in the data, but it may have difficulty with gradual changes. Although being adept at identifying conceptual changes across time, EDDM may have difficulty with noisy data. In addition, it takes 20 errors before it begins to detect drift. The key outcome of ACE is that if the concept is stationary, qt will not deteriorate below the interval's lowest endpoint. This enables ACE to detect concept changes quickly, whereas selecting a small window size could result in false positives. Only after getting $2W$ samples does ACE begin to identify drift. When the P -value is smaller than aw , STEPDP employs aw and ad as the two significance levels of and saves samples in short-term memory. For STEPDP, The classifier is then constructed using these examples and all variables are reset if the P -value becomes less than ad . When $no + nr \geq 2W$ is satisfied, STEPDP only begins to identify drift. The stored examples are eliminated if the P -value is greater than or equal to aw . This makes it possible for STEPDP to look at the difference in concept rapidly and precisely.

To evaluate if the difference between the two proportions is statistically significant, the McNemar's test statistic is utilized. If the test statistic exceeds a preset significance level, the difference is judged significant and it is decided that the two classifiers perform differently. If the test statistic is smaller than the critical value, the difference is not regarded as statistically significant, and it is assumed that there are few significant variations between the performances of the two classifiers as a consequence.

Table 3 presents results for drift detection in datasets using SVM and LSTM classifiers. The "Method set" column refers to the method used for drift detection. The cumulative predicted error rate and confidence interval are displayed in the "Error Rate" column. The "Nd" column shows the number of drift detections. The number of examples necessary to accurately identify drift is

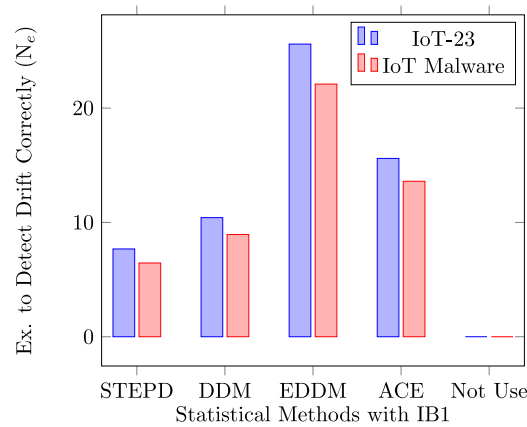


Fig. 8. IB1 Performance over required no. of samples for retraining.

displayed in the “Ne” column. The “Data” column indicates the dataset being used. The LSTM method seems to perform better in terms of both error rate and Ne, with lower values compared to the SVM method in both datasets. In the IoT-23 dataset, the STEP and DDM methods had the highest error rates among the SVM methods, while the EDDM and ACE methods had higher Ne values. For the LSTM method, the STEP and DDM methods had the lowest error rates, and the EDDM and ACE methods had lower Ne values. In the IoT-Malware dataset, all of the SVM methods had higher error rates compared to the LSTM method, with the EDDM method having the highest error rate among all methods. The DDM and ACE methods also had higher Ne values for the SVM method compared to the LSTM method. We note that IB1 classifier is slow to respond to all the statistical methods, since it needs most no of sample to retrain. Continuously monitoring the drift detection can help reduce the adaptation time effectively in case of EDM. This fact is depicted in Fig. 8.

Limitations: In summary, these findings imply that the LSTM technique outperforms the SVM method in these datasets in identifying concept drift. The error rates and Ne values across the various approaches within each classifier, however, fluctuate dramatically, highlighting how the methodology chosen may have a considerable impact on the effectiveness of the drift monitoring. On the basis of these findings alone, it is not feasible to draw definitive conclusions regarding whether concept drift occurred in the data or not. To properly assess the impact of concept drift on the performance of these models, it would be crucial to obtain additional information on the data used to train and evaluate the models, such as the time period over which the data was acquired and the degree of change in the data over time.

5. Conclusion & future work

Statistical drift detection methods are well-suited to applications where the data is constantly changing, such as in real-time streaming data (IIoT), because they can be updated as new data arrives and adapt to changes in the data distribution. Online classifiers are typically more efficient and can handle larger datasets than batch learning algorithms, but there may be more dataset shift if the data distribution dramatically shifts over time. In future many other tests and techniques can be applied to various different scenarios. On the basis of these findings alone, it is not feasible to draw definitive conclusions regarding whether concept drift occurred in the data or not. To properly assess the impact of concept drift on the performance of these models, it would be crucial to obtain additional information on the data used to train and evaluate the models, such as the time period over which the data was acquired and the degree of change in the data over time.

To evaluate the level of change in the data over time, we employ statistical tests. For instance, we compare the means or percentage of various data points gathered at various moments in time using tests. If the test finds a large difference between the proportions or false positive rate, it means that the data have changed significantly over time. The results in the figure show that the LSTM model, which is followed by the SVM, Naive Bayes, and IB1 models, has the best precision, recall, F1-score, and accuracy. The LSTM model has the lowest values for both metrics in terms of false positive and false negative rates, following the SVM model, the Naïve Bayesian model, and the IB1 model.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] IoT stats. 2023, URL <https://explodingtopics.com/blog/iot-stats>. (Accessed 01 jan 2023).
- [2] Kaspersky. Malware attacks on IoT devices in 2021–22. Kaspersky; 2022, URL <https://www.kaspersky.com/blog/iot-report-2022/>.
- [3] McAfee. Threat landscape for IoT devices. McAfee; 2020–2021, URL <https://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-july-2020.pdf>.
- [4] Gu S, Iusarczyk B, Hajizada S, Kovalyova I, Sakhibieva A. Impact of the covid-19 pandemic on online consumer purchasing behavior. *J Theor Appl Electron Commer Res* 2021;16(6):2263–81.
- [5] Nishida K, Yamauchi K. Detecting concept drift using statistical testing. In: *International conference on discovery science*. Springer; 2007, p. 264–9.
- [6] Klabjan D, Zhu X. Neural network retraining for model serving. 2020, arXiv preprint [arXiv:2004.14203](https://arxiv.org/abs/2004.14203).
- [7] Weiss K, Khoshgoftaar TM, Wang D. A survey of transfer learning. *J Big Data* 2016;3(1):1–40.
- [8] Chilakapati A. Concept drift and model decay in machine learning. In: *Data exploration*. 2020, URL <https://xplordat.com/2019/04/25/concept-drift-and-model-decay-in-machine-learning/>.
- [9] Gama J, Iliobait I, Bifet A, Pechenizkiy M, Bouchachia A. A survey on concept drift adaptation. *ACM Comput Surv* 2014;46(4):1–37.
- [10] Ditzler G, Roveri M, Alippi C, Polikar R. Learning in nonstationary environments: A survey. *IEEE Comput Intell Mag* 2015;10(4):12–25.
- [11] Lu J, Liu A, Dong F, Gu F, Gama J, Zhang G. Learning under concept drift: A review. *IEEE Trans Knowl Data Eng* 2018;31(12):2346–63.
- [12] Iwashita AS, Papa JP. An overview on concept drift learning. *IEEE Access* 2018;7:1532–47.
- [13] Demar J, Bosni Z. Detecting concept drift in data streams using model explanation. *Expert Syst Appl* 2018;92:546–59.
- [14] Lang CI, Sun F-K, Lawler B, Dillon J, Al Dujaili A, Ruth J, et al. One class process anomaly detection using kernel density estimation methods. *IEEE Trans Semicond Manuf* 2022;35(3):457–69.
- [15] Tennant M, Stahl F, Rana O, Gomes JB. Scalable real-time classification of data streams with concept drift. *Future Gener Comput Syst* 2017;75:187–99.
- [16] Gama J, Zliobaite I, Bifet A, Pechenizkiy M, Bouchachia A. Concept drift detection: A review. *ACM Comput Surv* 2010;42(4):43.
- [17] Gemaque RN, Costa AFJ, Giusti R, Dos Santos EM. An overview of unsupervised drift detection methods. *Wiley Interdiscip Rev Data Min Knowl Discov* 2020;10(6):e1381.
- [18] Sato DMV, De Freitas SC, Barddal JP, Scalabrín EE. A survey on concept drift in process mining. *ACM Comput Surv* 2021;54(9):1–38.
- [19] Amin M, Tanveer TA, Tehseen M, Khan M, Khan FA, Anwar S. Static malware detection and attribution in android byte-code through an end-to-end deep system. *Future Gener Comput Syst* 2020;102:112–26.
- [20] Amin M, Shehwar D, Ullah A, Guarda T, Tanveer TA, Anwar S. A deep learning system for health care IoT and smartphone malware detection. *Neural Comput Appl* 2020;1–12.
- [21] Gulcan EB, Can F. Unsupervised concept drift detection for multi-label data streams. *Artif Intell Rev* 2022;1–34.
- [22] Garcia S, Parmisano A, Erquiaga MJ. IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0). Zenodo; 2020, <http://dx.doi.org/10.5281/zenodo.4743746>.
- [23] Ding F. IoT malware. 2017, <https://github.com/ifting/iot-malware>.
- [24] Ding F, Li H, Luo F, Hu H, Cheng L, Xiao H, et al. DeepPower: Non-intrusive and deep learning-based detection of IoT malware using power side channels. In: *Proceedings of the 15th ACM Asia conference on computer and communications security*. 2020, p. 33–46.
- [25] Gama J, Medas P, Castillo G, Rodrigues P. Learning with drift detection. In: *Brazilian symposium on artificial intelligence*. Springer; 2004, p. 286–95.
- [26] Baena-García M, del Campo-vila J, Fidalgo R, Bifet A, Gavalda R, Morales-Bueno R. Early drift detection method. In: *Fourth international workshop on knowledge discovery from data streams*, vol. 6. 2006, p. 77–86.
- [27] Nishida K, Yamauchi K, Omori T. ACE: Adaptive classifiers-ensemble system for concept-drifting environments. In: *International workshop on multiple classifier systems*. Springer; 2005, p. 176–85.
- [28] Li W, Zhang Y, Sun Y, Wang W, Li M, Zhang W, et al. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Trans Knowl Data Eng* 2019;32(8):1475–88.
- [29] Tong X, Ozturk P, Gu M. Dynamic feature weighting in nearest neighbor classifiers. In: *Proceedings of 2004 international conference on machine learning and cybernetics (IEEE Cat. No. 04EX826)*, vol. 4. IEEE; 2004, p. 2406–11.